Lab 7: Image Steganography and LSB Forensics

Part 0 — Setup and evidence inventory

```
# Uncomment if needed, then run once:
# %pip install pillow stepic

from pathlib import Path
from PIL import Image
import hashlib

ASSET_DIR = Path("Lab7")
LOGO = ASSET_DIR / "compay_logo.png"
PROFILE = ASSET_DIR / "profile.png"
PYTHON_IMG = ASSET_DIR / "python.png"
OUTPUT_DIR = Path("Lab7_outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

for path in [LOGO, PROFILE, PYTHON_IMG]:
    if not path.exists():
        raise FileNotFoundError(f"Missing {path}. Check where you extracted Lab8.zip")
    with Image.open(path) as image:
        print(path.name, image.size, image.mode, path.stat().st_size, "bytes")
```

```
compay_logo.png (1024, 1024) RGBA 1029126 bytes
profile.png (1408, 768) RGBA 2424849 bytes
python.png (100, 100) RGBA 9427 bytes
```

Check-in 0: What does RGBA mean? Why might a forensic analyst calculate a file's SHA-256 hash?

It stands for Red, Green, Blue, and Alpha (transparency). Each pixel is represented by these four color/transparency channels. To generate a unique cryptographic checksum of the file to verify its integrity and detect any subsequent modifications.

Evidence workflow: The hashes recorded below are **baseline values**. You will reuse the same function in Part 5 to compare the original images with their stego-images, then verify that message extraction does not change a stego-image. A hash difference proves the file bytes differ; it does **not** prove steganography.

```python
def sha256_file(path):
    """Calculate the SHA-256 digest of a file's exact bytes."""
    with open(path, "rb") as file:
        return hashlib.sha256(file.read()).hexdigest()

# Save baseline hashes now; we will use these values again in Part 5.
original_logo_hash = sha256_file(LOGO)
original_profile_hash = sha256_file(PROFILE)

print("Original logo SHA-256:   ", original_logo_hash)
print("Original profile SHA-256:", original_profile_hash)
```

```
Original logo SHA-256:    64303ae8e227a463b9d1c9c27359d228bf0874c1284535f6f80f3e0802a6a6e0
Original profile SHA-256: 292d9a5bcf8b10017abc1e633a86ae0ddbac4e5c2156e84d6dbf5307b3df5850
```

Part 1 — Binary and LSB fundamentals

Worked example: set and read one bit

```python
def set_LSB(value, bit):
    """Return an integer with its least significant bit set to 0 or 1."""
    if bit not in ("0", "1"):
        raise ValueError("bit must be '0' or '1'")
    return (value & 254) if bit == "0" else (value | 1)

def get_LSB(value):
    return str(value & 1)

value = 150
for bit in ("0", "1"):
    changed = set_LSB(value, bit)
    print(value, format(value, "08b"), "→", changed, format(changed, "08b"), "LSB", get_LSB(changed))

assert set_LSB(150, "1") == 151
assert get_LSB(151) == "1"
```

```
150 10010110 → 150 10010110 LSB 0
150 10010110 → 151 10010111 LSB 1
```

Your turn — small modifications

1. Change the starting value to 201 and test setting its LSB to 0 and 1.
2. Explain why set_LSB(201, "1") does not change the value.
3. What is the **maximum numerical difference** between an original channel and its LSB-modified value?

```
# TODO: Run two examples with value = 201 and print binary before/after.
value = 201
for bit in ("0", "1"):
    changed = set_LSB(value, bit)
    print(value, format(value, "08b"), "→", changed, format(changed, "08b"), "LSB", get_LSB(changed))
# your code here
```

```
201 11001001 → 200 11001000 LSB 0
201 11001001 → 201 11001001 LSB 1
```

For value = 201, setting its LSB to 0 gives 200, while setting it to 1 keeps it 201 because its LSB is already 1. The maximum numerical difference between an original channel and its LSB-modified value is 1.

Part 2 — Calculate capacity before writing

```
def max_message_bytes(width, height, channels=4, terminator_bytes=1):
    capacity_bytes = (width * height * channels) // 8
    return max(0, capacity_bytes - terminator_bytes)

print("100×100 RGBA:", max_message_bytes(100, 100, 4), "message bytes")
print("100×100 RGB:", max_message_bytes(100, 100, 3), "message bytes")

payload = "TARGET:192.168.1.50"
required_bytes = len(payload.encode("utf-8")) + 1
print("Payload + terminator:", required_bytes, "bytes")
print("Pixels needed (supplied RGBA method):", required_bytes * 2)
assert required_bytes == 20
```

```
100×100 RGBA: 4999 message bytes
100×100 RGB: 3749 message bytes
Payload + terminator: 20 bytes
Pixels needed (supplied RGBA method): 40
```

Question: How many message bytes fit in a 100 × 100 RGBA image using this exact method? How would an RGB-only, one-LSB-per-channel method differ?

100×100 RGBA image can hold **4999 message bytes**, whereas an RGB-only image holds **3749 message bytes**. The RGB method provides less capacity because it lacks the alpha channel.

Part 3 — Manual LSB encoding and extraction

```python
# Guided reference test: import the supplied script without modifying it.
import sys
sys.path.insert(0, str(ASSET_DIR.resolve()))
from steganography_LSB import hide_message, extract_message

TEST_STEGO = OUTPUT_DIR / "python_test_stego.png"
hide_message(str(PYTHON_IMG), "HI", str(TEST_STEGO))
recovered = extract_message(str(TEST_STEGO))
print("Recovered:", recovered)
assert recovered == "HI"
```

```
Recovered: HI
```

Task 3A — Inspect the reference code

1. Why does the reference code call .convert("RGBA")?

   To ensure the image has four channels (Red, Green, Blue, Alpha) so that LSB data can be hidden uniformly across all of them.

2. Why are two pixels used per byte?

   Each pixel provides 4 LSBs (one per channel), so two pixels provide 8 bits (1 byte) total.

3. What does message += chr(0) accomplish?

   It adds a null/zero-byte terminator at the end of the message to signal where the message stops during extraction.

4. What happens if the payload exceeds image capacity? How could your code detect this *before* editing any pixels?

   It raises an error or overwrites/corrupts data. It can be detected beforehand by comparing the required message bytes (plus terminator) against the image's maximum capacity calculation.

5. Why is PNG preferable to JPEG for preserving exact LSB values?

   PNG uses lossless compression, keeping pixel values identical, whereas JPEG uses lossy compression, which alters pixel values and destroys hidden LSB data.

Task 3B — Implement a validated encoder (partial starter)

```python
def encode_checked(source_path, message, output_path):
    """Validate a short ASCII message, then call the supplied encoder."""
    source_path = Path(source_path)
    output_path = Path(output_path)
    if not source_path.is_file():
        raise FileNotFoundError(source_path)
    if not message:
        raise ValueError("Message must not be empty")
    if "\x00" in message:
        raise ValueError("Zero byte is reserved as a terminator")
    with Image.open(source_path) as image:
        available_pixels = image.width * image.height
    needed_pixels = 2 * (len(message.encode("utf-8")) + 1)
    # TODO 1: If needed_pixels exceeds available_pixels, raise ValueError.
    if needed_pixels > available_pixels:
        raise ValueError("Payload exceeds image capacity")
    # TODO 2: Call hide_message(...) with the three parameters.
    hide_message(str(source_path), message, str(output_path))
    # TODO 3: Return output_path.
    return output_path
    raise NotImplementedError("Complete TODOs before running")

# After implementing, uncomment:
LOGO_STEGO = encode_checked(LOGO, "TARGET:192.168.1.50", OUTPUT_DIR / "company_logo_stego.png")
print(LOGO_STEGO)
```

```
Lab7_outputs\company_logo_stego.png
```

Task 3C — Build a decoder

```python
def decode_manual(path):
    with Image.open(path) as image:
        pixels = list(image.convert("RGBA").getdata())
    decoded_bytes = bytearray()
    for index in range(0, len(pixels) - 1, 2):
        first, second = pixels[index], pixels[index + 1]
        # TODO: collect the eight LSBs from first + second
        bits = "".join(str(channel & 1) for channel in first + second)
        # TODO: convert the 8-bit string to an integer
        value = int(bits, 2)
        # TODO: stop if value == 0; otherwise append to decoded_bytes
        if value == 0:
            break
        decoded_bytes.append(value)
    # TODO: return decoded_bytes.decode("utf-8")
    return decoded_bytes.decode("utf-8")
    raise NotImplementedError("Complete the decoder")

# Test after completing both functions:
assert decode_manual(LOGO_STEGO) == "TARGET:192.168.1.50"
assert extract_message(str(LOGO_STEGO)) == "TARGET:192.168.1.50"
```

```
C:\Users\user\AppData\Local\Temp\ipykernel_3388\1120332739.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14
(2027-10-15). Use get_flattened_data instead.
  pixels = list(image.convert("RGBA").getdata())
```

Part 4 — Stepic: second implementation

```
ports = [80, 443, 3478, 5060]
port_text = ",".join(str(port) for port in ports)
port_bytes = port_text.encode("utf-8")
print("Message:", port_text)
print("Bytes:", port_bytes)
assert port_text == "80,443,3478,5060"
```

```
Message: 80,443,3478,5060
Bytes: b'80,443,3478,5060'
```

Task 4A — Encode and save

```
# If Stepic is installed:
import stepic
with Image.open(PROFILE) as source:
    rgb_image = source.convert("RGB")
    # TODO: call stepic.encode(rgb_image, port_bytes)
    stego_image = stepic.encode(rgb_image, port_bytes)
    # TODO: save returned image to OUTPUT_DIR / "profile_secret.png"
    output_path = OUTPUT_DIR / "profile_secret.png"
    stego_image.save(output_path)
    print("Saved to:", output_path)
```

```
Saved to: Lab7_outputs\profile_secret.png
```

Task 4B — Decode and verify

```
# TODO: Open profile_secret.png, call stepic.decode, normalize to text if needed.
with Image.open(OUTPUT_DIR / "profile_secret.png") as img:
    decoded = stepic.decode(img)
    recovered_port_text = decoded.decode("utf-8") if isinstance(decoded, bytes) else decoded

# TODO: assert recovered_port_text == port_text
assert recovered_port_text == port_text

# Print the recovered text.
print(recovered_port_text)
```

```
80,443,3478,5060
```

Analysis: Why is it important to verify extraction rather than assuming a successful save means the payload survived?

Because file saving or system operations can sometimes cause unexpected data corruption or formatting alterations without raising an explicit error. Verification ensures the hidden payload actually survived intact.

Part 5 — File integrity, visual, and pixel analysis

```python
def compare_images(original_path, stego_path):
    with Image.open(original_path) as original, Image.open(stego_path) as stego:
        a = list(original.convert("RGBA").getdata())
        b = list(stego.convert("RGBA").getdata())
        if original.size != stego.size:
            raise ValueError("Image dimensions differ")
    changed_pixels = sum(p != q for p, q in zip(a, b))
    changed_channels = sum(x != y for p, q in zip(a, b) for x, y in zip(p, q))
    return {
        "original_bytes": Path(original_path).stat().st_size,
        "stego_bytes": Path(stego_path).stat().st_size,
        "original_sha256": sha256_file(original_path),
        "stego_sha256": sha256_file(stego_path),
        "same_file_bytes": sha256_file(original_path) == sha256_file(stego_path),
        "changed_pixels": changed_pixels,
        "changed_channels": changed_channels,
        "total_pixels": len(a),
    }

print("Reference test:", compare_images(PYTHON_IMG, TEST_STEGO))
```

```
Reference test: {'original_bytes': 9427, 'stego_bytes': 9471, 'original_sha256': 'b1f47b31ac5fb11d45b17576d57775931191da73bec566b02074a119bb6700fd', 's
tego_sha256': '10aa54fdbce9fa694c1b71c484dba8eeda68aa66d4b84f4cb7629e11f26c22bd', 'same_file_bytes': False, 'changed_pixels': 6, 'changed_channels': 1
8, 'total_pixels': 10000}
C:\Users\user\AppData\Local\Temp\ipykernel_3388\40363051.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14
(2027-10-15). Use get_flattened_data instead.
  a = list(original.convert("RGBA").getdata())
C:\Users\user\AppData\Local\Temp\ipykernel_3388\40363051.py:4: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14
(2027-10-15). Use get_flattened_data instead.
  b = list(stego.convert("RGBA").getdata())
```

Task 5A — Reuse your baseline hashes

```python
# Task 5A — Complete after Parts 3 and 4.
# Uncomment and adapt these lines; keep the earlier baseline variables.
#
logo_stats = compare_images(LOGO, LOGO_STEGO)
profile_stego = OUTPUT_DIR / "profile_secret.png"
profile_stats = compare_images(PROFILE, profile_stego)

print("Original logo baseline matches current original:",
      original_logo_hash == sha256_file(LOGO))
print("Original logo vs. stego identical:",
      original_logo_hash == sha256_file(LOGO_STEGO))
print("Original profile vs. stego identical:",
      original_profile_hash == sha256_file(profile_stego))

print("Logo pixel comparison:", logo_stats)
print("Profile pixel comparison:", profile_stats)
#
# TODO: Calculate and print the percentage of changed pixels for each pair.
logo_changed_pct = (logo_stats["changed_pixels"] / logo_stats["total_pixels"]) * 100
profile_changed_pct = (profile_stats["changed_pixels"] / profile_stats["total_pixels"]) * 100

print(f"Logo changed pixels: {logo_changed_pct:.4f}%")
print(f"Profile changed pixels: {profile_changed_pct:.4f}%")
# TODO: In a Markdown cell, explain what your actual hash results demonstrate.
```

```
C:\Users\user\AppData\Local\Temp\ipykernel_3388\40363051.py:3: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14
(2027-10-15). Use get_flattened_data instead.
  a = list(original.convert("RGBA").getdata())
C:\Users\user\AppData\Local\Temp\ipykernel_3388\40363051.py:4: DeprecationWarning: Image.Image.getdata is deprecated and will be removed in Pillow 14
(2027-10-15). Use get_flattened_data instead.
  b = list(stego.convert("RGBA").getdata())
Original logo baseline matches current original: True
Original logo vs. stego identical: False
Original profile vs. stego identical: False
Logo pixel comparison: {'original_bytes': 1029126, 'stego_bytes': 816889, 'original_sha256': '64303ae8e227a463b9d1c9c27359d228bf0874c1284535f6f80f3e080
2a6a6e0', 'stego_sha256': 'a8634a1d37758640a45d211b802d569b981dcd22cfbf89b4d84a54b9738c8638', 'same_file_bytes': False, 'changed_pixels': 39, 'changed_
channels': 80, 'total_pixels': 1048576}
Profile pixel comparison: {'original_bytes': 2424849, 'stego_bytes': 1902331, 'original_sha256': '292d9a5bcf8b10017abc1e633a86ae0ddbac4e5c2156e84d6dbf5
307b3df5850', 'stego_sha256': '0c9913c5322e00d97d6bce575520598c805882bee17aa3819967f697f927dbad', 'same_file_bytes': False, 'changed_pixels': 43, 'chan
ged_channels': 74, 'total_pixels': 1081344}
Logo changed pixels: 0.0037%
Profile changed pixels: 0.0040%
```

Checkpoint: If two images look identical but their hashes differ, does that prove one contains a hidden message? Why or why not? What additional evidence would you examine?

No, a different hash only proves the files are not identical. To prove steganography, you

must find hidden data through extraction or analyze pixel-level patterns.

Task 5B — Verify extraction did not modify the stego-image

```
# Run after completing your manual decoder in Part 3.
before_extract = sha256_file(LOGO_STEGO)
recovered = decode_manual(LOGO_STEGO)
after_extract = sha256_file(LOGO_STEGO)

print("Recovered message:", recovered)
print("Before extraction:", before_extract)
print("After extraction: ", after_extract)
assert before_extract == after_extract, "Extraction unexpectedly changed the file!"
assert recovered == "TARGET:192.168.1.50"
#
# TODO: Repeat for profile_secret.png with stepic.decode(...).
profile_secret_path = OUTPUT_DIR / "profile_secret.png"
before_extract_profile = sha256_file(profile_secret_path)

with Image.open(profile_secret_path) as img:
    decoded = stepic.decode(img)
    recovered_profile = decoded.decode("utf-8") if isinstance(decoded, bytes) else decoded

after_extract_profile = sha256_file(profile_secret_path)

print("Recovered profile message:", recovered_profile)
print("Before extraction (Profile):", before_extract_profile)
print("After extraction (Profile):", after_extract_profile)
assert before_extract_profile == after_extract_profile, "Extraction unexpectedly changed the file!"
assert recovered_profile == port_text

# TODO: Record whether the hash changed and explain why this matters.
```

```
Recovered message: TARGET:192.168.1.50
Before extraction: a8634a1d37758640a45d211b802d569b981dcd22cfbf89b4d84a54b9738c8638
After extraction:  a8634a1d37758640a45d211b802d569b981dcd22cfbf89b4d84a54b9738c8638
Recovered profile message: 80,443,3478,5060
Before extraction (Profile): 0c9913c5322e00d97d6bce575520598c805882bee17aa3819967f697f927dbad
After extraction (Profile): 0c9913c5322e00d97d6bce575520598c805882bee17aa3819967f697f927dbad
```

Record whether the hash changed and explain why this matters.

The hash did not change, which proves that the extraction operation did not modify the file bytes, maintaining forensic integrity.

Task 5C — View the original and modified images

```
from IPython.display import display

def show_pair(original_path, stego_path):
    with Image.open(original_path) as original, Image.open(stego_path) as stego:
        print("Original:")
        display(original.copy().resize((300, 300)))
        print("Stego:")
        display(stego.copy().resize((300, 300)))

# TODO: show_pair(LOGO, LOGO_STEGO)
show_pair(LOGO, LOGO_STEGO)
# TODO: show_pair(PROFILE, OUTPUT_DIR / "profile_secret.png")
show_pair(PROFILE, OUTPUT_DIR / "profile_secret.png")
```

Original:



Stego:



Original:



Stego:



Visually indistinguishable.

Part 6 — Independent security analysis and reflection

LSB steganography conceals data within the least significant bits of image pixels, maintaining visual appearance, whereas encryption transforms plaintext into ciphertext while keeping the data structure visible. If an individual knows the extraction method, the hidden message can easily be read.

Both payloads were successfully recovered and verified using Python assertions that

matched “TARGET:192.168.1.50” and “80,443,3478,5060”. Image capacity depends on dimensions, channels, and terminators because total available bits dictate the storage ceiling, and a null terminator ($0x00$) is required to signal the end of the payload. PNG is superior to JPEG because its lossless compression preserves exact LSB values, whereas JPEG's lossy compression alters pixel data.

Our SHA-256 results showed that embedding payloads changed the file bytes and hashes of the stego-images, but extraction did not alter them. Different hashes do not prove steganography because any minor modification, such as metadata changes, alters the hash. Two limitations of LSB steganography are low payload capacity and vulnerability to detection, while two defensive steps include LSB statistical analysis and visual inspection. Finally, the fictional TARGET label and port list are not evidence of actual data theft because they are isolated, simulated lab artifacts.

Github

https://github.com/bnwzc/FSCT8561-Labs-Kuangda-Zhu